Nüwa: Enhancing MLIR Fuzzing with LLM-Driven Generation and Adaptive Mutation

Bocan Cao†
ACSS Lab, Northwest University
Xi'an, China
caobocan@stumail.nwu.edu.cn

Weiyuan Tong†
ACSS Lab, Northwest University
Xi'an, China
wytong@stumail.nwu.edu.cn

Zhanyong Tang*
PINE Lab , Northwest University
Xi'an, China
zytang@nwu.edu.cn

Zixu Wang
ACSS Lab, Northwest University
Xi'an, China
wangzixu1@stumail.nwu.edu.cn

Hao Huang
ACSS Lab, Northwest University
Xi'an, China
huanghao1289@stumail.nwu.edu.cn

Yuheng Yan
PINE Lab, Northwest University
Xi'an, China
2022117139@stumail.nwu.edu.cn

2025 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 979-8-3315-9587-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSME64153.2025.00055

***Abstract*—MLIR, a modular compiler framework, evolves quickly, with regular updates expanding its dialects and operations across LLVM versions and downstream projects. This fast development reduces the effectiveness of traditional fuzzing tools, which test only a small portion of dialects, require extensive manual work (e.g., nearly ten thousand lines of C++ code), and do not match the update speed of MLIR. To address these challenges, we propose NÜWA, the first LLM-based approach for MLIR fuzzing. NÜWA employs a two-phase strategy: first generating valid operations by encoding constraints into LLMs prompts, then synthesizing multi-operation test cases by learning inter-operation dependencies. To enhance operation coverage, it incorporates high-coverage cases from MLIR's test suite and uses LLM-driven mutations to boost diversity. A self-improvement mechanism enhances the prompts using feedback from high-quality test cases, improving the LLMs' understanding of MLIR's complex semantics. NÜWA demonstrates that the generation and mutation process can be fully automated via the intrinsic capabilities of LLMs (including in-context learning), while being applicable to MLIR's fast evolution. The experimental study shows that NÜWA outperforms the state-of-the-art tools MLIRSmith and MLIRod, detecting 2.9× more unique bugs and achieving 1.6× greater code coverage. To date, NÜWA has identified 55 bugs in the MLIR framework, with 18 confirmed or fixed.**

***Index Terms*—MLIR, Large Language Models, Fuzzing, Test Generation, Test Mutation**

I. Introduction

MLIR (Multi-Level Intermediate Representation) is a modular compiler infrastructure, designed to address software fragmentation, limited hardware heterogeneity support, and high development costs in domain-specific compiler construction [1]. Built on the principle of "little builtin, everything customizable", MLIR enables flexible extensions through fundamental constructs like operations, types, and attributes [2], empowering domain-specific abstraction layers (*Dialects*) [3]. Its multi-layered intermediate representations (IRs) have significantly advanced deep learning compiler technologies, optimizing neural network compilation workflows through integrations with frameworks such as XLA [4] and TensorFlow [5]. MLIR's adoption in industrial ecosystems (e.g., TPU compilers [6], ONNX [7]) demonstrates its leadership in heterogeneous compilation [8], [9].

However, MLIR's customizability introduces heightened risks of compiler defects. Like traditional compilers [10], [11], MLIR infrastructure may contain bugs that propagate to downstream compilers and applications, potentially causing critical failures [12]. Fuzzing, a proven technique for detecting compiler defects through program generation [13], thus becomes essential for ensuring MLIR's reliability.

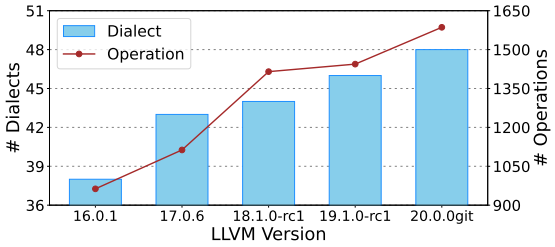


Fig. 1: The number of MLIR dialects and operations across the five major LLVM releases.

Version Evolution Challenges: The fast evolution of the MLIR framework poses a challenge for maintaining an effective and sustainable fuzz testing strategy. During the past year, MLIR has released 27 versions [1], each introducing significant extensions and modifications. To analyze the changes within MLIR, we quantify the number of operations and dialects across the last five major MLIR versions. As shown in Figure 1, the number of dialects and operations increased by 11.6% and 29.8%. Currently, 48 dialects and 1,444 operations have been developed internally. In addition, 33 public downstream

* Corresponding author.
† Co-primary authors, both authors contributed equally to this research.
ACSS Lab stands for Xi'an Key Laboratory of Advanced Computing and Software Security. PINE Lab stands for Shaanxi Key Laboratory of Passive Internet of Things and Neural Computing.

[1]As a subproject of the LLVM repository [14], MLIR is released in conjunction with LLVM

2576-3148/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSME64153.2025.00055

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

projects like CIRCT [15] have also each contributed multiple custom dialects [16]. This dynamic evolution renders existing test cases obsolete as they fail to exercise new language features, necessitating low-cost fuzzer adaptation strategies.

Limitations of Prior Work: While existing approaches like MLIRSmith [17] and MLIRod [18] demonstrate initial progress, three critical gaps persist: (1) *Narrow dialect coverage* - current tools support only around 20 dialects, inadequate for MLIR's expanding ecosystem; (2) *High maintenance overhead* - template-based generators require 10K+ lines of C++ code (e.g., MLIRSmith) and expert-level MLIR knowledge for gray-box adaptation; (3) *Update latency* - manual framework modifications cannot match MLIR's rapid release cadence, causing obsolescence. Through our empirical study, the 1,000 test cases generated by MLIRSmith for LLVM 16 are almost unusable when fuzzing the new MLIR version on LLVM 18.

Motivation: To address the dual challenges of MLIR's rapid evolution and the limitations of template-driven fuzzing, it's necessary to construct test cases in a more automated manner. Prior studies [19], [20] have demonstrated the strong code generation capabilities of LLMs [21] and significant success in testing compilers and language libraries. For MLIR fuzzing, we can provide operation constraints to prime LLM's in-context learning, generating test cases that conform to the syntax of MLIR. However, this approach of directly constructing multi-operation MLIR programs is inefficient, as LLMs must simultaneously process extensive specifications, infer both intra- and inter-operation constraints, manage complex multiple dependencies, including data, control, and structural dependencies, among others. This motivates our incremental generation, which first encodes domain operation specifications in the prompt to drive LLM in correctly generating all operations in MLIR, and then primes LLM to learn complex dependencies between operations to synthesize them into valid multi-operation cases. Additionally, LLM might fail to generate certain complex syntactic structures and specific operations in MLIR, due to lack of enough specialized knowledge [22]. To address this issue, we leverage LLM for automatic mutation to compensate for the limited coverage of test cases in LLM-based generation. A promising approach is to sew high-coverage cases into the generated test cases [23]. There are high-coverage cases available in the MLIR's test suite written by developers, achieving full coverage of all operations in MLIR (but maintaining a weak correlation with bug detection [24]). This drives us to leverage LLMs to synthesize high-coverage cases with generated cases, creating a more complex context that increases the likelihood of exposing potential bugs. Moreover, we also consider a feedback mechanism to further enhance the test generation/mutation capacity of LLM on the fly.

Our Approach: We propose NÜWA, the first MLIR fuzzing approach that unifies specification-driven generation and adaptive mutation via large language models (LLMs). With powerful code generation capabilities [21] of LLM, the generation and mutation process can be fully automated, thus departing from constructing IR by hand-craft. NÜWA consists of three key components: (1) *Two-phase Generation*: NÜWA adopts a two-phase generation approach that guides LLMs to analyze intra-operation and inter-operation constraints, generating valid test cases on the fly (Sec. III-C). (2) *Adaptive Mutation*: Seamlessly sew unit tests into generated test cases, thereby enhancing detection opportunities and effective coverage (Sec. III-D). (3) *Feedback-based Evolution*: Using high-quality test cases as feedback to enhance test generation and mutation prompt, thus guiding LLMs to produce more effective test cases (Sect. III-E).

In this paper, we demonstrate the capability of large language models to generate domain-specific intermediate representations. By leveraging LLMs, we establish an automated framework for generating MLIR programs to facilitate MLIR fuzzing. NÜWA enables the rapid and automated adaptation of new MLIR versions with minimal manual intervention, continuously uncovering bugs in the ever-evolving MLIR ecosystem. This approach facilitates the rapid integration of new versions, ensures reliability, and accelerates the adoption of emerging features, providing substantial value for the maintenance and evolution of the MLIR framework. In summary, our contributions are as follows.

- We present a two-phase generation approach and synthesis-based mutation via LLMs, enabling the automatic creation of valid and complex test cases while seamlessly adapting to evolving MLIR versions.
- We introduce a self-improvement mechanism that automatically improves LLM prompts, enabling LLMs to better grasp the intricate semantics and structure of MLIR programs to generate more valuable test cases.
- We conducted an extensive study for the comparison of NÜWA with state-of-the-art tools for MLIR fuzzing. Our result shows that NÜWA detected 2.9× more bugs and achieved 1.6× higher coverage than baseline methods. By now, 55 bugs have been reported, 2 already confirmed and 16 already fixed.

II. BACKGROUND

A. MLIR Compiler Infrastructure

MLIR is an SSA-based compiler framework that aims to simplify the creation, transformation, and optimization of intermediate representations. *Operation* is the foundational elements of MLIR, representing all entities and processes [25]. Everything from instructions to functions to modules is modeled as an operation in MLIR. Each operation can accept zero or more operands and produce zero or more result values, with structures defined using TableGen specifications [26]. These specifications enable standardized Static Single Assignment (SSA) representations [3]. *Dialect* is a core concept in MLIR, representing a logical grouping of operations that support a range of reusable abstractions. MLIR supports a variety of dialects; each dialect defines its own operations, types, and semantics, enabling the efficient representation of different abstractions, such as machine learning, and hardware description. Dialects are used to define and represent specific

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

operations within IR, i.e., MLIR program. MLIR allows a mix of various dialects that support different types within a single MLIR program. For example, in one program, the TOSA dialect for high-level tensor operations and the LLVM dialect for low-level hardware operations can coexist. *Pass* is the core mechanism for IR optimizations and transformations, enabling efficient code generation. The MLIR compiler invokes conversion passes to convert one dialect to another, as well as transformation passes that rewrite IR into a more efficient version.

B. MLIR Fuzzing

A primary challenge in MLIR testing is to produce valid and sufficiently varied programs that fully exercise different compiler components. Invalid programs are typically discarded in early compilation stages (e.g., preprocessing), whereas syntactically diverse inputs can improve code coverage and expose latent defects [12]. Existing MLIR fuzzing efforts encompass both generation-based [17] and mutation-guided approaches [27]. For example, MLIRSmith leverages a template-driven method to produce programs covering twelve dialects, while MLIRod enhances the test input diversity by mutating these programs. Recently, the state-of-the-art MLIRtracer [28] conducts directed MLIR fuzzing, relying solely on a generator for the TOSA dialect, but still involves hard-coded grammar rules. These techniques all share the common limitations of grammar-based fuzzing, namely limited flexibility and manual overhead.

C. Large Language Models

Large language models (LLMs) have demonstrated significant potential in areas such as code generation, text understanding, and dialogue systems [21], [29]–[31]. Models like OpenAI's GPT-4, trained on vast datasets containing hundreds of billions of tokens [32], have advanced natural language processing (NLP), enabling them to execute complex tasks without task-specific fine-tuning. To address knowledge gaps, *few-shot learning* [33] incorporates example-based prompts, improving output accuracy and enabling the model to solve complex problems [19], [20].

LLMs have emerged as powerful tools for code-aware test generation, particularly suited to compiler fuzzing challenges. LLMs' capabilities in text generation and comprehension enable the following three key advantages for MLIR testing. First, LLMs can generate syntactically valid MLIR dialects by learning operation specifications and MLIR semantics, reducing manual template construction. Second, unlike template-based generators limited to predefined dialects, LLMs infer tensor attributes and data flow constraints between operations, ensuring more diversity while preserving semantic consistency. Third, *few-shot learning* [33] enhances LLMs' understanding of MLIR's complex syntactic structures, improving the efficiency of both generation and mutation processes. In summary, by leveraging the intrinsic capabilities of LLMs, including in-context learning, the generation and mutation process for MLIR testing can be fully automated while remaining adaptable to MLIR's rapid evolution.

III. Approach

A. Overview

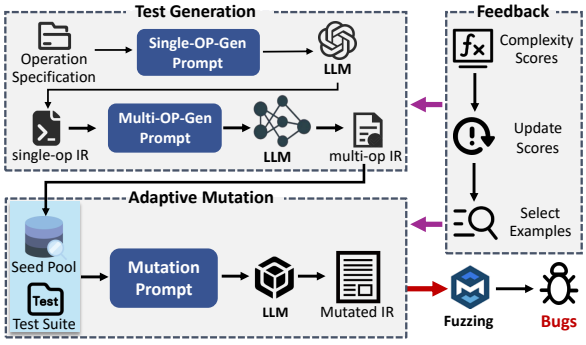


Fig. 2: Overview of NÜWA.

Figure 2 illustrates the overall architecture of NÜWA, which comprises three main components: test generation, adaptive mutation, and a feedback loop. During the generation phase, NÜWA employs a two-phase approach using LLMs to create a suite of MLIR programs covering various dialects that serve as test cases (Sec.III-C). Specifically, NÜWA first creates single operations (i.e., single-op IR) based on operation specifications and then synthesizes multiple valid operations to construct complex multi-op IR. To further enhance operation coverage, NÜWA implements an LLM-driven mutator (Sec. III-D) that automatically integrates high-coverage IR ingredients from the test suite with generated multi-op IRs. Moreover, during both the generation and mutation phases, NÜWA introduces a feedback mechanism by providing effective cases to LLMs for contextual learning, thereby enhancing its generation capabilities (Sec. III-E). For each test case, NÜWA randomly selects a pass from the MLIR compiler infrastructure to transform or convert it. If the compiler crashes, it indicates a potential bug and is reported to the developers.

B. Conversational Interaction

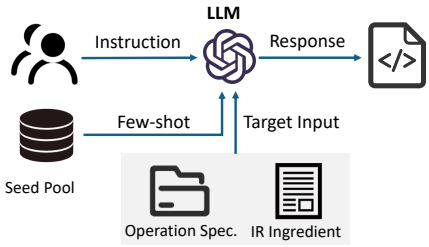


Fig. 3: Conversational interaction with LLMs

NÜWA is designed to communicate with the large language models to guide test case generation and mutation in a conversational manner. As shown in Figure 3, each task prompt consists of three parts: *instruction*, *few-shot*, and

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

target input. We first specify the application scenario, target output, and task details in the *instruction*. The *target input* and *few-shot* together form the context for the task, driving LLMs response for the current task. The *target input* primarily includes operation specifications and test suite IR ingredients extracted and formatted from the MLIR infrastructure. The *few-shot* includes three examples [34] selected from the seed pool by the feedback algorithm (see Sec. III-E), with each example consisting of a seed and its corresponding input at the time of generation, enabling LLMs to perform contextual learning [35]. NÜWA automatically verifies the syntactical correctness of the seeds and provides feedback to LLMs until no undefined behavior is present or a preset interaction limit is reached.

Algorithm 1: The workflow of generator

```
1 Function Generator():
2     foreach dialect ∈ Dialects do
3         foreach spec ∈ dialect do
4             ops ← Generate(spec, dialect.config)
5         InitCompScore(dialect)
6         for i ← 0 to dialect.config.N do
7             examples ← SelectExamples(dialect, 3)
8             IR ← Generate(ops, examples)
9             UpdateCompScore(IR, dialect)
```

C. Two-phase Generation

Directly generating multi-op cases is challenging due to LLMs' limited MLIR domain knowledge and inability to grasp complex task contexts [36]. Specifically, generating a multi-op case requires LLMs to simultaneously process all operation specifications, infer both intra- and inter-operation constraints, and handle multiple dependencies of data, control flow, and structure. This significantly increases the computational complexity of the attention mechanism [37] and poses a risk of task failure if any operation contains syntactical errors. Therefore, we propose an LLM-based two-phase generation method: first, generate single-op IRs for each specification; then synthesize them into a multi-op case. This approach significantly reduces context length at each step and ensures that the case composition is based on correct operation understanding. Algorithm 1 delineates the workflow of NÜWA's generator. NÜWA sequentially generates test cases per dialect (Line 2). For each dialect, single-op IRs are first generated from operation specifications (Lines 3-4). Subsequently, NÜWA generates multi-op IR (Lines 6-8) by instantiating the task prompt with single-op IRs and examples selected using the feedback-driven algorithm (Sec.III-E) to steer LLM-driven generation.

1) Extraction Operation Specifications: In this work, operation specifications provide contextual corpora for LLMs, guiding them to generate single-op IR under intra-operation constraints. These specifications follow a structured format that enables the model to understand each operation's requirements and constraints. Each specification comprises the operation name, description, input and output types, attributes, and other constraints, which are automatically extracted and parsed using a script. Figure 4 presents a standardized operation specification for the operation "tosa.floor", including definitions, descriptions, input and output types, attributes, and additional details.

```
{"OpName": "tosa.floor",
  "Description": "Elementwise floor operation",
  "Input": [
  {"Name":"input1", "Dim_range":[0,4], "Type":"float"}],
  "Result": [
  {"Name":"output", "Dim_range":[0,4], "Type":"any"}]
}
```

Fig. 4: Specification of "tosa.floor" operation

Instruction -
Assume you are an expert in the MLIR infrastructure. Based on Specification of the "toas.floor", identify its syntactic rules and generate an IR - free of undefined behavior. I will first provide the expected output format, followed by the context of the target input, and finally present several few-shot examples.

Target Input -
< Operation Specification as shown in Figure 4 >

Response -

```
module {
  func.func @floor(%arg0:tensor<4xf32>)->tensor<4xf32> {
    %0="toas.floor"(%arg0): (tensor<4xf32>)->tensor<4xf32>
    return %0 : tensor<4xf32>
  }
}
```

Fig. 5: With the operation specification, a general prompt framework is instantiated and used to generate single-op IR.

2) Single Operation Generation: At this stage, we instantiate the general framework in Figure 3 into the "Single-OP-Gen Prompt". As shown in Figure 5, we take the generation process of the "tosa.floor" operation as an example. First, in the *instruction* part, we select *input type* as the operation specification and provide detailed task instructions. Next, through the feedback loop (Sec. III-E), we choose examples from the seed pool to build *few-shot*; these examples belong to the same dialect as the target operation. Each example demonstrates the process of generating a single-op IR from an operation specification (*input*) to its corresponding *response*. The examples guide LLMs to generate output that meets the expected format and reinforce the mapping from the low-level operation specification to the high-level operation program. Finally, NÜWA fills in the target operation specification shown in Figure 4 in the *target input* part and uses the complete prompt to drive the generative LLM to produce test cases for the target operation IR. The final output generated by LLMs is presented in the *response* part; NÜWA then extracts and verifies the case's syntax correctness and provides interactive feedback to LLMs until a valid test case is produced or the preset iteration limit is reached.

3) Multi-operation IR Synthesis: In the multi-op IR synthesis phase, we instantiate the general framework in Figure 3

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

Instruction -
Assume you are an expert in the MLIR infrastructure. Based on single-op IRs of "tosa.avg_pool2d", "tosa.const", "tosa.rsqrt", "tosa.add", "tosa.sigmoid" and "tosa.transpose_conv2d" following the syntax rules and data flow dependencies, generate an IR with no syntax errors ...

Target Input -

```
%0 = tosa.avg_pool2d %arg0 ... : (...) ->
           tensor<1x3x30x30xf32>
%const = "tosa.const"() {...} : () -> tensor<1x1xf32>
%output = tosa.rsqrt (%input1):(tensor<?x?xf32>) ->
           tensor<?x?xf32>
%0 = "tosa.add"(...) : (tensor<12x6xf32>, ...) ->
           tensor<12x6xf32>

...
```

Response -

```
module {
  func.func @main(%arg0:tensor<1x3x32x32xf32>,...) {
    %pool=tosa.avg_pool2d %arg0...:(tensor<1x3x32x32xf32>)
            -> tensor<1x3x30x30xf32>
    %const = "tosa.const"() {...} : () -> tensor<1x1xf32>
    %avg = "tosa.add" %pool,%const:(...) ->
            tensor<1x3x30x30xf32>
    ...
    return %avg, %transpose, ...: tensor...
  }
}
```

Fig. 6: Instantiate a general prompt framework for multi-op IR generation.

as the "Multi-OP-Gen Prompt", which drives the generative LLM to synthesize single-op IRs into multi-op IRs with complex structures and dependencies under inter-operation constraints. As shown in Figure 6, the synthesis process for six operations, including "tosa.avg_pool2d" and "tosa.const", is demonstrated. Initially, NÜWA selects *input type* in the *instruction* part as the single-op IRs and provides detailed task instructions to guide LLMs under MLIR syntax rules constraints in integrating multiple single-op IRs into a complete program. Subsequently, through the feedback loop (Sec.III-E), examples are selected to construct *few-shot*; each example contains multiple complete single-op IRs (*input*) along with their correct synthesis results (*response*). Under the influence of these few-shot examples, the generative LLM is prompted to produce IR results in the correct format and to reinforce the mapping from individual operation syntax rules to the data and control flow among multiple operations. Finally, according to the configuration (Sec.III-E2), NÜWA fills the *target input* part with the complete IR source code for each target operation. The generated IR is shown in *response*, NÜWA automatically extracts the case, and checks its syntax correctness and target operation, and feeds the check results back to LLMs through the interactive template, continuing until successful generation or the preset interaction limit is reached.

D. Adaptive Mutation

To compensate for the limited coverage of test cases in LLM-based generation, we employ LLMs for synthesized-based mutation by combining high-coverage cases from the test suite with generated test cases to produce more complex structures. Specifically, we employ two methods in the mutation implementation: invocation synthesis and inline synthesis.

Algorithm 2: The workflow of mutator

```
1 Function Mutator():
2     foreach s ∈ SeedPool do
3         instances ← CandidatePool
4         examples ← SelectExamples(s, 3)
5         mutant ← Mutate(s, instances, examples)
6         UpdateCompScore(mutant)
```

Invocation synthesis integrates IR ingredients (organized as functions) by invoking them within a main function, thereby preserving modular structure and inter-function dependencies and enhancing the detection of cross-function optimization strategies. Inline synthesis, in contrast, inlines all operations from IR ingredients into a single function, creating a more complex contextual environment that increases the likelihood of exposing latent defects in complex data and control flows.

Algorithm 2 delineates the workflow of NÜWA's mutator. For each generated case in the seed pool (Line 2), we first select a predetermined number of IR instances from the formatted test suite candidate pool (Line 3), then sample examples through the feedback-driven algorithm (Sec.III-E). Finally, we instantiate the task prompt with both the case and selected suite IR and examples to guide LLMs in synthesizing mutations (Line 5).

Instruction -
Assume you are an expert in the MLIR infrastructure. Based on single-func IRs of "@splat", "@slice" and "@tanh", follow the syntax rules and data/control flow dependencies, to merge these functions using invocation synthesis and inline synthesis without undefined behavior, ensuring they reside within the same module ...

Target Input -

```
func.func @splat() -> tensor<1x1x1xi32> {
  %splat = tosa.const() ... : () ->tensor<4x5x6xi32>
  ...
}
func.func @slice(tensor<4x5x6xi32>) ->tensor<1x1x1xi32> {
  %slice = tosa.slice %splat ... : -> tensor<1x1x1xi32>
  return %slice : tensor<1x1x1xi32>
}
```

Response -

```
module {
  func.func @main(%arg0:tensor<1x3x32x32xf32>,...) {
    %splat = "tosa.const"() ... : () -> tensor<4x5x6xi32>
    %slice = call @slice(%splat) : -> tensor<1x1x1xi32>
    ...
    return %slice, %tanh, ... : tensor...
  }
func.func @slice(tensor<4x5x6xi32>) ->tensor<1x1x1xi32> {
    ... }
... }
```

Fig. 7: Instantiate a general prompt framework for adaptive mutation.

For the mutation, NÜWA first statically analyzes all test suites in the MLIR infrastructure and classifies them by syntax type into a candidate pool. Then, we instantiate the general framework in Figure 3 into the "Mutation Prompt" to drive the mutation LLM to perform the corresponding tasks. As shown in Figure 7, the adaptive mutation process for three functions, including "@splat" "@slice" and "@tanh" is demonstrated.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: The feedback based on complexity score

```
1  Function InitCompScore(SeedPool):
2      for s_i ∈ SeedPool do
3          s_i.score ← Complexity(s_i)
4      SortByScores(SeedPool)
5  Function SelectExamples(SeedPool, N):
6      examples ← arg max(SeedPool.score, N)
7      return examples
8  Function UpdateCompScore(NewCase, SeedPool):
9      NewCase.score ← Complexity(NewCase)
10     CasePool ← NewCase
11     SortByScores(SeedPool)
```

NÜWA first selects *input type* as single-func IR in the *instruction* part. Guiding LLMs to perform adaptive mutation on multiple single-func IRs under the guidance of syntax rules, control relationships, and data dependencies. Then, through the feedback loop (Sec.III-E), examples are selected to construct the *few-shot* for this task. Each example includes multiple single-func IRs (*input*) and their correct synthesized mutation results (*response*). Finally, for each function to be mutated, NÜWA analyzes its syntax type and parameters, based on the configuration, automatically selects a preset number of IRs from the initialized test suite, extracting their source code to populate the *target input* part. NÜWA then uses the complete prompt to drive LLMs. The mutated IR is shown in *response*, and with each interaction, NÜWA automatically extracts the test case, checks its syntax correctness, target function, and data/control flow relationships, continuing until successful mutation or the preset interaction limit is reached. The data and control flow checks ensure that these functions are not simply placed in the same module without actual data or control relationships.

E. Feedback Loop

Although zero-shot [19] method has achieved success, it are limited in their effectiveness for generating complex MLIR programs due to the lack of domain-specific knowledge. To address knowledge gaps, *few-shot learning* [33] incorporates example-based prompts, improving output accuracy and enabling the model to solve complex generation problems. To facilitate *few-shot learning* for LLMs, NÜWA provides a feedback mechanism for generation and mutation. This mechanism provides LLMs with highly complex test cases as examples, helping them grasp the intricate semantics and structure of MLIR programs, to generate more valuable test cases.

1) Feedback Based on Complexity Scores: To pick up the representative example as feedback for LLMs, we assess the complexity score of test case using the following formula:

$$score = \frac{(N_d \times N_o)^{N_t}}{len(IR)} \tag{1}$$

where N_d, N_o, and N_t refer to the number of dialects, operations, and unique data types in IR, respectively. $len(IR)$ refers to the total character length in a test case. In particular, for cases with similar complexity, those with shorter character lengths receive higher scores because they reduce the context length required by LLMs and enhance the efficiency of constructing complex test cases. Consequently, test cases with more dialects, operations, and data types, but shorter IR lengths are given higher scores.

Initially, NÜWA generates a minimal set of seeds through zero-shot generation. Subsequently, the feedback loop is activated to construct a few-shot prompt framework, continuously enhancing the quality of generation and mutation. Algorithm 3 demonstrates the workflow of our complexity score-driven feedback algorithm. When the feedback loop begins, NÜWA calculates the complexity score for existing test cases in the seed pool and ranks them according to their complexity score (Lines 2-4). Specifically, for subsequent new test cases generated or mutated by LLM, NÜWA also calculates their complexity score and adds them to the seed pool (Lines 9-11). During subsequent generation or mutation, selects three IRs (by default) with the highest scores as examples in the few-shot prompt framework (Figure 3). This feedback mechanism guides LLM to generate more complex test cases, with its performance outperforming zero-shot, static example, or random feedback approaches. We have demonstrated by our ablation experiments (Sec. V-C). Note that in the generation phase, examples are selected from IRs of the same dialect, while in the mutation phase, syntax-compatible IRs are chosen at random.

2) Configuration Adjustment: Throughout the test case construction process, NÜWA maintains a configuration dictionary to control key parameters, such as models, interaction frequencies, and the proportion of dialects used in test case construction, among others. This dictionary serves as a flexible mechanism for adjusting the test case generation and mutation process, enabling precise management. Based on the error counts of each dialect in the test results, the configuration can be dynamically adjusted to prioritize dialects with more mutations or higher error rates. This approach enhances testing focus, improves efficiency in identifying compiler issues, and ensures continuous optimization of the generation and mutation processes for better results across varied contexts.

IV. Experimental Setup

A. Research Questions

We evaluate the effectiveness of our approach using the following research questions (**RQs**):

- **RQ1:** Can NÜWA detect previously unknown bugs in the MLIR compiler infrastructure? (Sec. V-A)
- **RQ2:** How is the effectiveness of NÜWA compared with state-of-the-art fuzzers on the MLIR infrastructure? (Sec. V-B)
- **RQ3:** Are all components of NÜWA contributing positive improvements to its final effectiveness? (Sec. V-C)
- **RQ4:** How do different parameter settings and experimental configurations affect NÜWA 's performance? (Sec. V-D)

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

B. Implementation

During the seed generation phase, we first generate single-op IR programs for 34 dialects and 825 operation specifications, and additionally collect 46 dialects and 1,426 operations from the test suite for adaptive mutation. Subsequently, based on this, we utilize a multi-op generation and adaptive mutation strategy to extensively construct a large number of test cases. NÜWA supports custom configurations for each dialect, including model selection, generation weights, and other specific parameters in the configuration file (Sec. III-E2).

For test case generation and mutation, we employed GPT-4o [38] model as the LLM, which represents the most advanced model available at the time of our study. The prompt generation is implemented in Python, and the interaction with GPT-4o [38] is run via its online Applications Program Interface (API). This API-based approach eliminates the need for local hardware deployment, without requiring fine-tuning. The cost of API usage is low, averaging less than $0.05 per test case.

C. Experimental Configurations

To address RQ1, NÜWA is executed to fuzz the latest version of the MLIR compiler infrastructure (commit ID: bab01ae) to expose previously unknown bugs. To answer RQ2 and RQ3, we conduct extensive experimental evaluations and ablation studies using the revision bec839. The deep learning models generated by NNsmith are converted into MLIR-compatible formats using the onnx-mlir frontend tool. For both MLIRSmith and MLIRod, we use the recommended parameter settings in their work. In particular, MLIRod initializes its seed pool with the MLIRSmith, as done in their paper. For NÜWA and its variants in comparative experiments, the seed count (N) is set to 500, including IRs generated from operation specifications in the generation phase and those further synthesized and mutated with test suite IRs in the mutation phase.

All our experiments are conducted on a machine with 62 cores (Intel(R) Xeon(R) Gold 6238R CPU @ 2.20GHz), 376GB RAM, and Ubuntu 20.04.6 LTS. To ensure sufficient testing, each experiment is conducted with a default time budget of 24 hours, and each experiment is repeated five times to reduce the impact of randomness.

D. Compared Work

We compare NÜWA with three state-of-the-art baseline techniques:

- **MLIRSmith** [17]: This approach randomly constructs programs by syntax-guided program template instantiation.
- **MLIRod** [18]: This approach utilizes dependency-directed mutation rules to direct the MLIR testing process toward enhancing operation dependency coverage.
- **NNSmith** [39]: One of the most advanced fuzzing generators for deep learning compilers, whose generated ONNX computation graphs are converted into MLIR-compatible formats for indirect MLIR testing.

E. Evaluation Metrics

We evaluate using the following metrics:

- **Number of Detected Bugs.** Fuzz testing may trigger multiple crashes, which are deduplicated based on their error message and submitted to developers after verification. Notably, developers may fix bugs without updating issue reports [17]; thus, we verify their presence in subsequent commits, classifying them as static fixes if absent.
- **Number of Covered Dialects and Operations.** We collect the number of dialects and operations included in the test cases to evaluate the comprehensiveness of the fuzz testing. A greater coverage of dialects and operations increases the likelihood of detecting edge cases and potential bugs.
- **Code Coverage** Code coverage is one of the key factors in accurately evaluating test coverage. To accurately compare the source code coverage, we use llvm-cov [40] and gcov [41] tools to collect relevant data and measure the line and branch coverage of the MLIR source code.
- **Generation Efficiency.** For comparison, with the same seed count (N), we record the total number of interactions with LLMs during the generation or mutation phases for both NÜWA and its variants, as well as the number of failures, to evaluate the impact of various components of NÜWA on overall generation efficiency.

V. EVALUATION

A. Bug Finding

TABLE I: Summary of detected bugs

Dialect	Conv.	Trans.	Dialect	Conv.	Trans.
tosa	6	6	affine	–	6
linalg	–	5	scf	1	7
amdgpu	–	1	arith	–	3
transform	–	2	spirv	–	1
quant	–	1	pdl_interp	1	–
llvm	–	5	cf	–	2
others	5	3			

In our experiments, each detected crash bug is accompanied by the corresponding compilation command and backtrace information indicating the failure location. Based on this information, we filter out duplicate bugs and report only the unique ones to the official repository. Additionally, this information allows us to ensure that all reported issues originate from the MLIR framework.

Bug Statistics. In general, we reported 55 crash bugs detected by NÜWA. Among the bugs we found, 18/16 have been confirmed/fixed by project maintainers. Others are awaiting developer responses. Table I categorizes report bugs by triggering components. Of these, 42 are transformation bugs and 13 are conversion bugs. We find that most of these crash on a specific dialect: out of the 55 reported bugs, 47 are triggered by dialect-specific passes, while the remaining 8 were unrelated to the dialect and are categorized as ”others”.

Bug Example. Figure 8 illustrates a test case that triggers a functional bug, causing a crash when the

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

```
#map = affine_map<(d0, d1) -> (d0 + d1)>
spirv.func @main(...) -> ...{
  affine.for %arg1 = 0 to 1 {
    %4 = affine.apply #map(%arg3, %arg5)
  }
  affine.for %arg1 = 0 to 1 {
    %5 = affine.load %alloc_0 : memref<1x32x32x8xf32>
  }
  ...
}
```

Fig. 8: Program snippet for triggering bug #116042.

`-affine-loop-fusion` pass is used. This pass invokes the `promoteIfSingleIteration` function to promote the loop body of an "Affine.for" operation to its containing block. The bug occurs due to a flaw in the `promoteIfSingleIteration`, which made an incorrect assumption that there would always be a surrounding "builtin.func" operation. As a result, when this assumption is violated, the function attempted to access a non-existent parent operation or block, resulting in a segmentation fault. This bug is resolved by removing the incorrect assumption. In particular, this bug is introduced due to erroneous assumptions made during the integration of the function registration and construction mechanisms in the new version (that is, the MLIR framework in LLVM 18). NÜWA detects and reports this bug in LLVM 19, and is subsequently fixed in LLVM 20. We investigated 16 fixed bugs and found that 8 bugs are introduced by such version changes.

B. Comparison with Prior Work

TABLE II: Comparison with existing work and NÜWA variants

Experiment	Tools	# Bugs	Coverage	
			# Dialects	# Operations
Prior Work	MLIRSmith	16	12	172
	MLIRod	18	21	343
	NNSmith	26	23	278
NÜWA Variants	NÜWA-Gen	38	34	625
	NÜWA-Mut	47	36	520
	NÜWA	**56**	**43**	**734**

We evaluated NÜWA and compared its performance with existing techniques. Table II summarizes the performance of NÜWA along with several state-of-the-art approaches during a 24-hour fuzz testing session, with the primary metrics being the number of bugs detected and the coverage of dialects and operations. As shown, NÜWA detects significantly more bugs than the other techniques. Specifically, it detects 3.5× and 2.94× more bugs than MLIRSmith and MLIRod, respectively, demonstrating its superior capability within the MLIR infrastructure. Figure 9(c) further illustrates the bug detection trends during the testing period, confirming that NÜWA consistently leads to the discovery of unique bugs. Moreover, NÜWA achieves broader coverage by exercising 2.58× and 1.55× more dialects than MLIRSmith and MLIRod, respectively, as well as 3.68× and 1.81× more operations. This expanded coverage not only underpins its robust bug detection efficiency, but also exposes significant locality issues in MLIRSmith and MLIRod, whose limited coverage results in overly concentrated and repetitive detections, thereby reducing overall efficiency.

Furthermore, we investigate the factors contributing to NÜWA's superior performance by analyzing the evolution of coverage metrics in the MLIR compiler infrastructure during the fuzz testing experiment. Figures 9(a) and (b) illustrate the temporal trends in line and branch coverage for both NÜWA and existing techniques in a 24-hour testing period. Although existing techniques rapidly reach coverage saturation, NÜWA is able to continuously maintain coverage growth throughout the 24-hour period. Specifically, the first 8-hour window contributes the most coverage, i.e., 86%, while the later three 4-hour windows are still able to contribute 5%, 3%, and 5% coverage respectively, demonstrating the effectiveness of NÜWA. In total, NÜWA achieves a final line coverage of 83,202, which is 1.53× higher than that of MLIRSmith and MLIRod, and a final branch coverage of 49,245, corresponding to a 1.6× improvement relative to the other techniques.

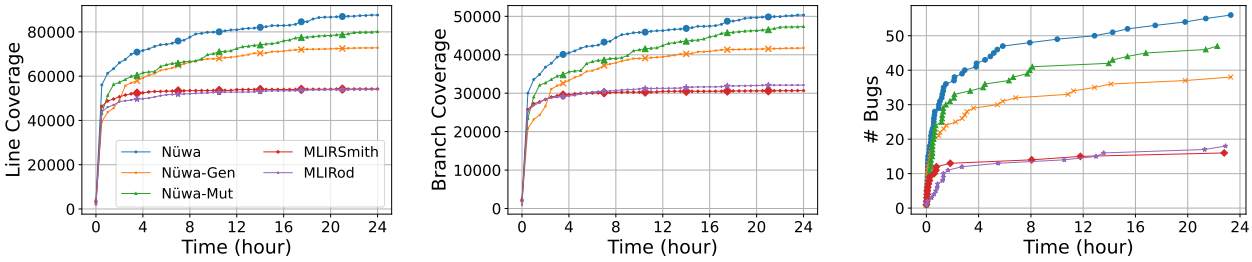
Overall, fuzz testing based on MLIR IR derived from high-level source code generated by NNSmith is suboptimal, as these programs are designed according to the characteristics of the source language rather than MLIR, resulting in limited diversity and reduced coverage. In contrast, NÜWA outperforms MLIRSmith and MLIRod across all metrics. This advantage stems primarily from the fact that the latter generate MLIR programs via meticulously crafted templates, thereby covering only a limited number of dialects and operations and struggle to keep pace with MLIR updates. Conversely, NÜWA encompasses nearly all existing dialects and most operations in MLIR with minimal update overhead, markedly enhancing the effectiveness of fuzz testing.

C. Ablation Study

In summary, NÜWA comprises three components: 1) an LLM-based two-phase generator that produces raw test IRs (Sec.III-C); 2) a mutator that synthesizes mutations between raw IRs and suite IRs (Sec.III-D); and 3) a feedback loop that optimizes the generation and mutation process (Sec. III-E). Next, we examine the contributions of each component.

We first evaluate the effectiveness of the generator and mutator by comparing two NÜWA variants. The NÜWA-Gen variant disables the adaptive mutation module, solely generating N seeds via the two-phase process, whereas NÜWA-Mut disables the generation module, which produces N seeds by applying mutations to test suite cases. The lower half of Table II (Variants part) indicates that the full NÜWA outperforms two variants in bug detection and coverage. Specifically, compared to NÜWA-Gen and NÜWA-Mut, NÜWA detects 1.47× and 1.2× more bugs, covers 1.27× and 1.2× more dialects as well as 1.17× and 1.4× more operations. An interesting observation is that the NÜWA-Mut detects more bugs despite covering fewer operations than NÜWA-Gen, demonstrating that mutation can produce higher-quality test cases. Figures 9 show

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.



(a) Line coverage trend (b) Branch coverage trend (c) Bug trend

Fig. 9: Coverage and bug trend

TABLE III: Ablation Study

Variants	#Generation			#Mutation		
	interactions	**failures**	**time(s)**	**interactions**	**failures**	**time(s)**
NÜWA-direct	1010	215	15184	\	\	\
NÜWA-0-shot	910	195	10327	\	\	\
NÜWA-fixed	475	80	6263	\	\	\
NÜWA-random	301	33	2415	660	180	7434
NÜWA	**264**	**32**	**2368**	**280**	**50**	**5813**

the growth trend of line coverage, branch coverage and bug detection count within 24 hours. We can see that the blue curve is always higher than the green and orange ones in the three sub-figures of Figures 9. This demonstrates that NÜWA consistently outperforms NÜWA-Mut and NÜWA-Gen in terms of code coverage growth and the efficiency in detecting bugs. In particular, from Figure 9, we can see that both NÜWA-Gen and NÜWA-Mut significantly surpass MLIRSmith and MLIRod in terms of code coverage and bug detection capacity. This suggests the effectiveness and necessity of the generator and mutator components.

In addition, we evaluate the effectiveness of the two-phase generation approach and feedback loop module. In NÜWA-direct variant, we directly generate multi-OP IRs from operation specifications, bypassing the single-op IR generation step in the two-phase process. Additionally, we design three NÜWA variants, i.e., NÜWA-0-shot, NÜWA-fixed, and NÜWA-random, providing no feedback or different feedback contents. NÜWA-0-shot uses the zero-shot prompt, relying solely on the inherent capabilities of the LLM without any examples. NÜWA-fixed and NÜWA-random provide few-shot learning examples. NÜWA-fixed uses a fixed set of examples, whereas NÜWA-random selects test cases with the same dialect randomly from a seed pool as examples.

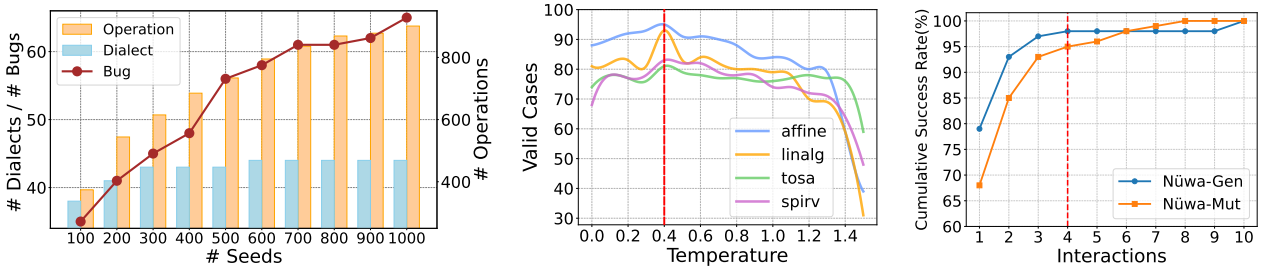
We evaluate the performance of NÜWA and four variants by comparing three metrics: the total number of interactions, the number of failures, and the duration time required to generate or mutate 100 test cases. As shown in Table III, we can see that NÜWA-direct requires significantly more interactions and time than the two-phase generation algorithm, and it experiences a multiplicative increase in failures. This underscores the robust capability of the two-phase generation algorithm in addressing complex code generation challenges. In the feedback loop evaluation, NÜWA completes the same tasks in the shortest time with the fewest interactions. Notably, we omit the mutation results of NÜWA-0-shot and NÜWA-fixed from the table due to their very poor mutation performance. Without the guidance of representative examples, they often fail or get stuck in indefinite loops in handling complex mutation tasks. This indicates that the feedback loop is crucial to the accuracy and efficiency of both generation and mutation. It also demonstrates the significant advantage of our approach in continuously refining prompts through high-quality examples introduced by the feedback mechanism, thereby enhancing LLM capabilities in real time compared to various alternative prompt structures.

D. Parameter Sensitivity

We investigate the impact of seed count on performance, with the results presented in Figure 10(a). To be specific, the x-axis represents the number of test cases, and the y-axis shows the number of covered dialects, operations, and detected bugs. As the figure shows, an increase in the number of test cases leads to greater dialect and operation coverage, as well as a higher number of detected bugs. This suggests that the number of test cases is a crucial factor in the effectiveness of MLIR fuzzing.

We set different values for the generation task to examine the impact of temperature on LLMs. This experiment involves four commonly used dialects: affine, linalg, tosa, and spirv, with 100 test cases generated for each dialect. Figure 10(b) plots the number of valid test cases versus temperature, where the x-axis represents temperature values and the y-axis represents the valid case count. The results show that generation performance peaks at a temperature of 0.4 and declines sharply when the temperature exceeds 1.0. This phenomenon is attributable to the temperature parameter's control over output randomness and diversity. As the temperature increases, the

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.



(a) Impact of seeds on NÜWA.

(b) Impact of temperature on LLMs.

(c) Impact of interactions on LLMs.

Fig. 10: Parameter Sensitivity

model gradually favors less probable tokens, yielding more varied and creative results. However, for high-precision tasks such as code generation, outputs must remain rigorous and logically consistent. Thus, lower temperature settings provide a better balance between accuracy and diversity, ensuring more precise and controllable outcomes.

Figure 10(c) presents the experimental statistics regarding the sensitivity of LLMs interactions limit. In the experiment, we record the cumulative success rates for different interaction counts, ranging from 1 to 10, for both the NÜWA-Gen and NÜWA-Mut variants. Each point on the curve represents the proportion of test cases constructed within the corresponding number of interactions shown on the x-axis. The results indicate that approximately 95% of the tasks are completed within 4 interactions with LLMs. This observation suggests that if a task remains incomplete after 4 interactions, it may be too complex and beyond the current model's generative capacity. Thus, we set the interaction limit for both generation and mutation to 4 to conserve time and token costs.

VI. DISCUSSION

A. Generality of NÜWA

Our approach exhibits broad generality and scalability, leveraging any LLMs while continuously improving performance as models evolve. Moreover, with minimal modifications to the prompt framework and the collection of relevant corpora, it can be effectively applied to fuzz testing a wide range of both traditional and deep learning compilers and libraries, ensuring rapid version compatibility.

B. Threats to Validity

Internal Threats. The primary source of internal threats lies in the construction and implementation of NÜWA, for which we have conducted rigorous code reviews and testing.

External Threats. The main external threat to validity arises from the evolving versions of MLIR. To accommodate new versions, NÜWA can automatically update MLIR's operation specifications and test suite. This enables the generation of test cases that comply with the new version's syntax rules, allowing rapid adaptation to new versions and testing with minimal cost. Additionally, we only employed GPT-4o in the implementation and evaluation of NÜWA, leading to another external validity. To demonstrate the ability of other LLMs in MLIR program generation, we also tested the effectiveness of MLIR fuzzing with two recently released state-of-the-art models, i.e., DeepSeek-V3 [42] and Gemini-2.0 [43], with the results listed in Table IV. As shown in the figure, all three models exhibit comparable performance in bug detection and coverage, owing to NÜWA 's deep understanding of the MLIR framework and effective utilization of model capabilities. In particular, to enhance the performance, it is essential to determine the optimal parameters when applying a new model to achieve peak performance, since different models exhibit varying sensitivity to parameters (e.g., temperature).

TABLE IV: Implementations with Different LLMs

LLMs	# Bugs	# Dialects	# Operations
DeepSeek-V3	56	44	720
Gemini-2.0	53	44	703
GPT-4o	56	43	734

Construct Threats. The primary construct threat to the validity of NÜWA lies in its implementation and configuration. To address this, we conducted multiple rounds of experiments to ensure the correctness of its implementation and the optimal configuration of parameters.

C. Future Work

Currently, NÜWA generates test cases for each dialect; however, defects have been detected only in 13 dialects and some passes, indicating an uneven distribution of defects. A promising solution is to adjust seed energy allocation in real time based on feedback from historical data and coverage information [44], [45]. Adjusting LLM generation scheme based on feedback during testing is our future work.

As MLIR rapidly evolves, the syntax of some new dialects becomes increasingly complex. Yet, pretrained LLMs often lack sufficient understanding of these intricacies, making direct code generation via prompts and few-shot learning challenging. Fine-tuning is a promising solution [46]. In the future, we will improve test case generation for complex operations by building a knowledge base for fine-tuning LLMs.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

VII. Related Work

Generation-based Fuzzing. Generative fuzzers synthesize test programs from scratch using language grammar and predefined templates, foregoing reliance on existing cases. Typical approaches involve hard-coded grammar rules [11], user-supplied grammar specifications [47], or intermediate representations (IR) to generate code for various downstream languages. Methods such as Csmith [11], YARPGen [48], and MLIRSmith [17] have advanced test generation and innovated in managing undefined behavior (UB) alongside overall strategies. Nonetheless, these approaches still face limitations in error detection efficiency and coverage [49], while the high maintenance cost of hard-coded rules and templates further complicates undefined behavior management.

Mutation-based Fuzzing. Mutation-based fuzzers enhance compiler testing by applying targeted modifications to seed programs, thereby uncovering vulnerabilities and improving code coverage [13]. Their mutations span simple insertions/deletions to complex transformations such as code splitting and concatenation [10], [50]. For instance, FLUX [24] synthesizes IR snippets using dual rules to probe LLVM optimizations, and MLIRod [27] employs target-dependent rules guided by an operation dependency graph (ODG) and operation dependency coverage (OD coverage) to enhance MLIR testing. Nonetheless, mutation-based fuzzing still contends with challenges in maintaining semantic consistency and managing undefined behavior [13], [51].

LLM-based Fuzzing. LLMs, with their powerful language processing capabilities, have shown great potential in code generation, text understanding, and related tasks [21], [29]–[31], becoming essential for enhancing software quality and security. Approaches like TitanFuzz [19] employs a zero-shot framework with mask mutation to boost coverage and uncover novel bugs, while PromptFuzz [52] utilizes coverage-guided prompt mutation to improve branch coverage. FuzzGPT [20] leverages historical bug-triggering programs to guide LLMs in producing high-quality test cases with superior bug discovery rates. These advancements underscore the growing efficacy of LLMs in automating fuzz testing and reinforcing software security, particularly in compiler testing [53]–[55].

VIII. Conclusion

The rapid evolution and flexibility of MLIR pose significant challenges for maintaining effective, low-cost fuzzing across its frequent updates. NÜWA addresses this by harnessing the robust comprehension abilities of large language models to generate test cases, moving beyond traditional grammar-based methods that rely on manually encoded rules. Through a two-phase generation process—first creating valid operations and then synthesizing multi-operation cases—and an adaptive mutation strategy guided by operation constraints, NÜWA delivers a fully automated fuzzing framework. This approach achieves wide test coverage and strong bug-finding capabilities, adapting seamlessly to new MLIR versions with little to no manual intervention. By offering a scalable and adaptable solution, NÜWA aims to transform fuzzing practices for rapidly evolving compiler frameworks. Beyond MLIR, NÜWA 's design offers promising extensibility to mainstream compilers and deep learning libraries, paving the way for broader applications in software testing.

IX. Data Availability

NÜWA has been open-sourced at GitHub [56].

Acknowledgement

We would like to thank the anonymous reviewers for their insightful comments and suggestions. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62372373 and in part by the Shaanxi Province "Engineers + Scientists" Team Building Program under Grant 2023KXJ-055.

References

[1] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: A compiler infrastructure for the end of moore's law," *arXiv preprint arXiv:2002.11054*, 2020.

[2] K. Majumder and U. Bondhugula, "Hir: An mlir-based intermediate representation for hardware accelerator description," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2023, pp. 189–201.

[3] T. Jin, G.-T. Bercea, T. D. Le, T. Chen, G. Su, H. Imai, Y. Negishi, A. Leu, K. O'Brien, K. Kawachiya *et al.*, "Compiling onnx neural network models using mlir," *arXiv preprint arXiv:2008.08272*, 2020.

[4] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 2–14.

[5] "Mlir dialects and utilities for tensorflow, tensorflow lite and xla." https://github.com/tensorflow/tensorflow/tree/master/tensorflow/compiler/mlir/, accessed 2024.

[6] P. Hu, M. Lu, L. Wang, and G. Jiang, "Tpu-mlir: A compiler for tpu using mlir," *arXiv preprint arXiv:2210.15016*, 2022.

[7] "Onnx-mlir," https://onnx.ai/onnx-mlir/, accessed 2024.

[8] "Tpu-mlir," https://github.com/sophgo/tpu-mlir, accessed 2024.

[9] "Triton," https://github.com/triton-lang/triton, accessed 2024.

[10] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," *ACM Sigplan Notices*, vol. 49, no. 6, pp. 216–226, 2014.

[11] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*, 2011, pp. 283–294.

[12] J. Chen, J. Patra, M. Pradel, Y. Xiong, H. Zhang, D. Hao, and L. Zhang, "A survey of compiler testing," *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–36, 2020.

[13] H. Ma, "A survey of modern compiler fuzzing," *arXiv preprint arXiv:2306.06884*, 2023.

[14] "Llvm-project," https://github.com/llvm/llvm-project, accessed 2024.

[15] "Circt:circuit ir compilers and tools," https://github.com/llvm/circt, accessed 2024.

[16] "Users of mlir," https://mlir.llvm.org/users/, accessed 2024.

[17] H. Wang, J. Chen, C. Xie, S. Liu, Z. Wang, Q. Shen, and Y. Zhao, "Mlirsmith: Random program generation for fuzzing mlir compiler infrastructure," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 1555–1566.

[18] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, "Fuzzing mlir compiler infrastructure via operation dependency analysis," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1287–1299.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.

[19] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 423–435.

[20] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt," *arXiv preprint arXiv:2304.02014*, 2023.

[21] OpenAI, "Chatgpt," 2022, accessed: 2025-01-09. [Online]. Available: https://openai.com/blog/chatgpt

[22] C. Cummins, V. Seeker, D. Grubisic, M. Elhoushi, Y. Liang, B. Roziere, J. Gehring, F. Gloeckle, K. Hazelwood, G. Synnaeve *et al.*, "Large language models for compiler optimization," *arXiv preprint arXiv:2309.07062*, 2023.

[23] V. Le, C. Sun, and Z. Su, "Finding deep compiler bugs via guided stochastic program mutation," in *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, ser. OOPSLA 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 386–399. [Online]. Available: https://doi.org/10.1145/2814270.2814319

[24] E. Liu, S. Xu, and D. Lie, "Flux: Finding bugs with llvm ir based unit test crossovers," in *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, September 2023, pp. 1061–1072.

[25] N. Katel, V. Khandelwal, and U. Bondhugula, "Mlir-based code generation for gpu tensor cores," in *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, 2022, pp. 117–128.

[26] "Llvm: Tablegen," LLVM (online), available from https://llvm.org/docs/TableGen/, accessed: 2020-07-01.

[27] "Mlirod," https://github.com/tju-chenyaosuo/MLIRod, accessed 2024.

[28] W. Tong, Z. Wang, Z. Tang, J. Fang, Y. Zhang, and G. Ye, "Directed testing in mlir: Unleashing its potential by overcoming the limitations of random fuzzing," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2288–2310, 2025.

[29] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[30] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[31] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[32] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large language models: A survey," *arXiv preprint arXiv:2402.06196*, 2024.

[33] L. Huang, P. Zhao, H. Chen, and L. Ma, "Large language models based fuzzing techniques: A survey," *arXiv preprint arXiv:2402.00350*, 2024.

[34] C. Yang, Y. Deng, R. Lu, J. Yao, J. Liu, R. Jabbarvand, and L. Zhang, "Whitefox: White-box compiler fuzzing empowered by large language models," *arXiv preprint arXiv:2310.15991*, 2023.

[35] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[36] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[37] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[38] "Gpt-4o." https://openai.com/index/hello-gpt-4o.

[39] J. Liu, J. Lin, F. Ruffy, C. Tan, J. Li, A. Panda, and L. Zhang, "Nnsmith: Generating diverse and valid test cases for deep learning compilers," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 530–543.

[40] "llvm-cov," https://www.llvm.org/docs/CommandGuide/llvm-cov.html, accessed 2024.

[41] "gcov," 2023. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Gcov.html

[42] "Deepseek v3: Advanced ai language model." https://deepseekv3.org/.

[43] "Gemini." https://deepmind.google/models/gemini/.

[44] W. Xu, S. Kashyap, C. Min, and T. Kim, "Designing new operating primitives to improve fuzzing performance," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 2313–2328.

[45] J. Li, B. Zhao, and C. Zhang, "Fuzzing: a survey," *Cybersecurity*, vol. 1, pp. 1–13, 2018.

[46] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *ICLR*, vol. 1, no. 2, p. 3, 2022.

[47] C. Holler, K. Herzig, and A. Zeller, "Fuzzing with code fragments," in *21st USENIX Security Symposium (USENIX Security 12)*, 2012, pp. 445–458.

[48] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for c and c++ compilers with yarpgen," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–25, 2020.

[49] G. Ye, T. Hu, Z. Tang, Z. Fan, S. H. Tan, B. Zhang, W. Qian, and Z. Wang, "A generative and mutational approach for synthesizing bug-exposing test cases to guide compiler fuzzing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1127–1139.

[50] C. Sun, V. Le, and Z. Su, "Finding compiler bugs via live code mutation," in *Proceedings of the 2016 ACM SIGPLAN international conference on object-oriented programming, systems, languages, and applications*, 2016, pp. 849–863.

[51] E. Nagai, A. Hashimoto, and N. Ishiura, "Reinforcing random testing of arithmetic optimization of c compilers by scaling up size and number of expressions," *IPSJ Transactions on System and LSI Design Methodology*, vol. 7, pp. 91–100, 2014.

[52] Y. Lyu, Y. Xie, P. Chen, and H. Chen, "Prompt fuzzing for fuzz driver generation," *arXiv preprint arXiv:2312.17677*, 2023.

[53] P. Liu, J. Liu, L. Fu, K. Lu, Y. Xia, X. Zhang, W. Chen, H. Weng, S. Ji, and W. Wang, "Exploring {ChatGPT's} capabilities on vulnerability management," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 811–828.

[54] H. Dhulipala, A. Yadavally, and T. N. Nguyen, "Planning to guide llm for code coverage prediction," in *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*, 2024, pp. 24–34.

[55] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.

[56] "nvwa," https://github.com/anonymity88/nvwa, 2024.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 11:04:14 UTC from IEEE Xplore. Restrictions apply.